DATA550 Module 6 Cheatsheet: GNU Make and Vim

GNU Make

Introduction

Motivation for Make

- Helps us build automated reports after modularizing code

Installing Make

Mac

- Type `which make` in terminal to see if you have make
- If not, run `xcode-select --install` in terminal

Windows

- Go to https://chocolatey.org/install and follow the website instructions
  - In Step 1, to open an administrative shell, go to Windows search and type `Windows Powershell`. Right click the app and click `Run as administrator`
  - Follow the other steps.
- Once Chocolately is installed, close and reopen `Windows Powershell`.
  - Check that Chocolately installed by typing `choco`
- Use Chocolately to install make by running `choco install make`
  - Check that make installed by opening a Git Bash Terminal and typing `which make`

Writing rules

Writing rules in make

- First, create a plain text file called a Makefile (can do this In terminal by typing `touch Makefile`)
  - A Makefile is a way to document dependencies in code
  - We write rules inside the Makefile
- Rule - a set of commands used to build an object called a `target` . A rule consists of a
  - `target` - the object we want to build

- `prequisites` - different pieces of code or objects in the directory we need to build the target
- `action` - what to do with the prerequisites to build the target
- Rule syntax:

```
target: prerequisites
    action
```

  - Note: need to indent the action using the `tab` key. Spaces will not work!

- Example Rule:

```
output/data_clean.rds: code/00_clean_data.R vrc01_data.csv
    Rscript code/00_clean_data.R
```

- To build the object, type `make <target_name>` in terminal e.g. `make output/data_clean.rds`
- Note: If you get the error `*** missing separator` , it's probably because you have spaces rather than tabs before the action

Build logic in Make

- Make is smart about what objects to make. It will not make them if not necessary.
- First, checks, `Does the target exist?`
  - If not, make the object.
  - If yes, go to the next question.
- Second, checks, `Have the prerequisities changed?`
  - If yes, make the object. Otherwise, Make outputs the message `<target> is up to date.`

Writing multiple, dependent rules

- If a prerequisite changes, make knows to rebuild all downstream objects that depend on the prerequisite, whether directly or indirectly
- eg. In this rule, we include a previous target (`output/data_clean.rds`) as a prerequisite for a new target, `output/table_one.rds.` If any of the prerequisites for `output/data_clean.rds` change, Make will also rebuild `output/table_one.rds` even if the dependency is not explicitly stated

```
output/table_one.rds: code/01_make_table1.R output/data_clean.rds
    Rscript code/01_make_table.R
```

Writing a clean rule

- A target doesn't have to be a file
- We can create a target that has no prerequisites

- The `clean` rule is helpful when working on reproducible workflows. It consists of actions to remove files. e.g.

```
clean:
    rm output/*.rds && \
    rm output/*.png
    (&& is used to chain commands and only runs if prior actions successfully
executed)
```

- Running `make clean` in terminal executes the action association with clean

PHONY targets in Make

- If we have a file called `clean` in our directory, Make will think the `clean` target is referring to the file and not do anything, outputting the message `clean is up to date`

- In this case, we need to declare `clean` as a `PHONY`. This lets Make know not to treat `clean` as on object but rather a shortcut name to execute an action, which should be executed whenever called

```
.PHONY: clean
clean:
    rm output/*.rds
```

Another example using PHONY targets

- Sometimes we want to update several pieces of the code at once. Instead of typing `make` for each object individually, we can create a PHONY target that lists all the objects we want to build as prerequisites.

- Since the objects are prerequisites, Make will build them if they are not up to date.

```
.PHONY: descriptive analysis
descriptive analysis: output/table_one.rds output/scatterplot.png
```

Grouped targets in Make

- Grouped targets can be used when a single action generates multiple targets. This lets Make knows it only needs to run the action once to get all the targets.

- Grouped target is specified by putting `&` after the last target

```
target1 target2 target3 &: prerequisities
    action
```

e.g.

```
output/both_models.rds output/both_tables.rds&: \
    code/03_models.R output/data_clean.rds
    Rscript code/03_models.R
```

```
.PHONY: regression_analysis
regression_analysis: output/both_models.rds output/both_tables.rds
```

Rendering the report using Make

- Create a rule to make the report

```
hiv_report.html: code/04/render_report.R \
    hiv_report.Rmd descriptive_analysis regression_analysis
    Rscript code/04_render_report.R
```

- Build report by typing `make hiv_report.html` in terminal
- Note: if you just run `make` in terminal, it will build the first target listed in the Makefile. Therefore, if you put the rule for the report first in the Makefile, you can build the report using the shortcut `make`

Using environment variables in rules

- In Make, we reference an environment variable using the syntax $`{VARIABLE}`
- Instead of hardcoding the names of objects/files that depend on the the value of `WHICH_CONFIG`, we can reference their names using environment variables
    - e.g. replace `hiv_report_config_default.html` -> `hiv_report_config_${WHICH_CONFIG}.html`

Final note on Make actions

- Actions may require executing multiple commands. However, they may not run in the order of the commands listed

```
output/table_one.rds: code/01_make_table1.R
    Rscript code/00_clean_data.R (&& \ )
    Rscript code/01_make_data.R
```

- If they needed to be run in order due to dependencies, it is necessary to chain them using `&&`

Vim

Basic Vim Operations

Motivation and introduction to vim

- Vim - a completely text-based editor (no buttons to click)
- Motivation
    - vim is useful if you need to do remote computing

    - sometimes git will open a vim session
- Opening and closing Vim
    - Open vim, type `vim` in terminal
    - Exit a Vim session `:q`

Saving files in vim

- Three modes of vim
    - `default mode` (can't insert text)
    - `insert mode` (when we want to write text)
    - `visual mode` (allows you to select text(
- Switching modes
    - To switch to default mode, press `Escape` key
    - To switch to insert mode, press `i`
    - To switch to visual mode, press `v`
- Default mode
    - we give commands, which start with `:`
    - To save a file, type `:w <file_name>` in default mode
    - if file already has a name, just use `:w`

Exiting vim with/out saving files

- In default mode (press `Escape` to enter default mode),
    - `:wq` - save file and close vim
    - `vim  <file>` - open a file in vim
    - `:q!` - close a file without saving files

Shortcuts

Moving around in vim

- Cannot use mouse to move cursor. Instead we need to move the cursor using the keyboard (e.g. arrows)
- Shortcuts for moving around in vim used in default mode
    - `G` - moves to last line
    - `gg` - move to first line
    - `$` - move to end of line
    - `^` - move to beginning of line
    - `{` - move up a paragraph

    - `}` move down a paragraph
- By default, insert mode allows you to start typing text wherever the cursor was last. It can be helpful to specify where we want to insert text. Shortcuts below essentially help to first move cursor and then enter insert mode

    - `I` - append text at beginning of line
    - `A` - insert text at end of line
    - `o` - insert text on new line

Removing characters in vim

- The basic way to remove character is to enter insert mode, and press `delete` key. However, this can be inefficient if want to remove a lot of text.

- Shortcuts for deleteing characters in default mode,

    - `x` - removes characters one a time after the cursor
    - `dw` - removes words one at a time
    - `u` - undo changes
    - `dd` - removes a whole line at a time
    - `D` - delete all text after cursor in the line

Finding and replace text in vim

- Find text In default mode

    - `/<word_to_find>` - highlights first instance of word appearing after the cursor
    - Press `Enter`, then pressing `n` will move to next occurrence of word, and `N` will move to previous occurrence of word

- Replace text in default mode

    - Global replacement: `:%s/<word_to_replace>/<new_word>/`

    - Local replacement:

        - Go to visual mode by typing `v`
        - Select lines to search and run the same command `:%s/<word_to_replace>/<new_word>/`